VERIQTA

The Cybersecurity Home Lab Handbook

Build. Practice. Investigate. Defend.
Real Skills for a Safer Digital World.

- KALI LINUX
- VIRTUALBOX
- NMAP
- WIRESHARK
- WAZUH
- OWASP JUICE SHOP
- FIREWALLS
- LINUX



- ✅ Step-by-Step Lab Setup
- ✅ Essential Tools and Commands
- ✅ Real-World Security Practice
- ✅ Threat Detection and Investigation
- ✅ System Hardening
- ✅ Hands-On Projects
- ✅ Build Your Cybersecurity Portfolio

BEGINNER FRIENDLY
PRACTICAL PROJECTS
REAL TOOLS
CLEAR EXPLANATIONS
PRODUCTION THINKING

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

01 Understanding a Cybersecurity Home Lab

VERIQTA

1. What is a Cybersecurity Home Lab?

A cybersecurity home lab is a safe, isolated environment on your personal computer where you can install and practice with different operating systems, security tools and vulnerable applications.

Key Points

- You create a virtual environment.
- You practice real security tasks.
- You can break things and learn safely.
- It does not affect your main computer.
- You control the entire lab.

Why Do Engineers Build a Home Lab?

- ✓ Learn by doing, not just reading.
- ✓ Understand how attacks and defenses work.
- ✓ Learn security tools and techniques.
- ✓ Practice troubleshooting real problems.
- ✓ Build hands-on skills for cybersecurity, DevOps, and cloud engineering.
- ✓ Prepare for jobs, interviews, and certifications.
- ✓ Experiment safely without risk to real systems.

2. Blue Team vs Red Team (and Defensive Security)

Blue Team (Defensive)



- Protects systems and data.
- Detects threats.
- Investigates security events.
- Responds to incidents.
- Uses tools like firewalls, SIEM and log analysis.

Red Team (Offensive)



- Simulates real attackers.
- Finds weaknesses.
- Performs authorized testing.
- Helps improve security.
- Uses tools like Nmap, Metasploit and web testing tools.

Defensive Security



- Focuses on preventing, detecting and responding to attacks.
- Includes both technology and processes.
- Essential for protecting real-world systems.

3. How a Home Lab Supports Multiple Fields

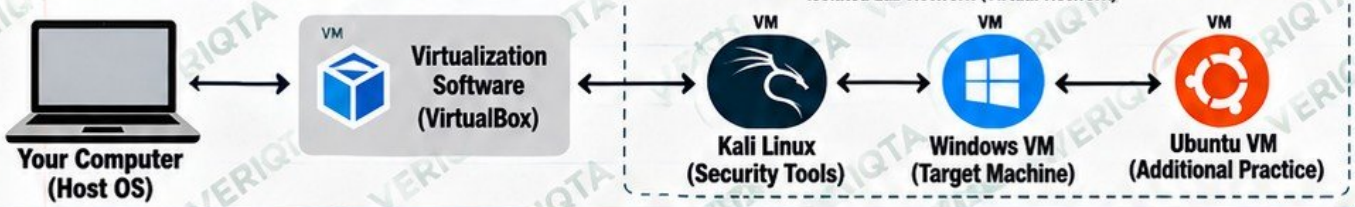
Cybersecurity	Learn security tools, threat detection, incident response and system hardening.
DevOps	Understand secure infrastructure, automation and monitoring.
Cloud Engineering	Practice cloud tools, identity, networking and security configurations.
General IT Skills	Improve Linux, networking, virtualization and troubleshooting.

4. Authorized Testing and Isolation

Always test only what you own or have permission to test.

- ✓ Use virtual machines and isolated networks.
- ✓ Do not scan or attack real systems.
- ✓ Keep your home lab separate from your personal and work environment.
- ✓ Follow legal and ethical guidelines.
- ✓ Use intentionally vulnerable machines for practice.

5. Example Home Lab Setup (High Level)



02 Hardware, Software, and Lab Requirements

1. Minimum and Recommended Hardware

Component	Minimum (Basic Lab)	Recommended (Better Performance)
CPU	4 cores (dual core with hyper-threading)	6–8 cores (multi-core CPU)
RAM	8 GB	16 GB or more
Storage	100 GB SSD	256 GB SSD or more

Key Points

- ✓ More CPU and RAM allow you to run multiple virtual machines smoothly.
- ✓ Use an SSD for better performance.
- ✓ You can start with the minimum requirements and upgrade later.
- ✓ A 64-bit processor is required for most virtualization software.

2. Choosing Your Host Operating System

Windows (Recommended for Beginners)

- User-friendly interface.
- Wide software support.
- Works well with VirtualBox and VMware.
- Good for beginners.

Linux (Good for Security Practice)

- Lightweight and flexible.
- Popular among security professionals.
- Works well with VirtualBox and KVM.
- Allows you to practice real Linux administration.

macOS (Good Performance)

- Stable and reliable.
- Works well with VirtualBox and Parallels (on Apple Silicon).
- Good performance.
- Useful for multi-platform learning.

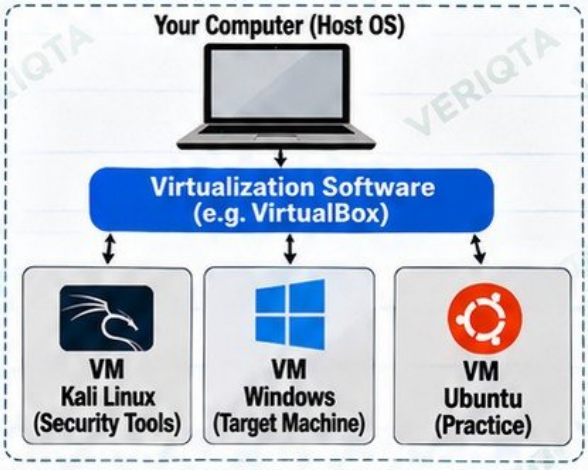
3. What is Virtualization and a Virtual Machine?

Virtualization

Virtualization allows you to run multiple operating systems on one physical computer by creating virtual machines (VMs).

Virtual Machine (VM)

A virtual machine is like a separate computer inside your computer. Each VM has its own operating system, files, and applications.



4. Free Tools for a Beginner Home Lab

VirtualBox	Free and open source virtualization software. Works on Windows, Linux and macOS.
Kali Linux	A popular Linux distribution for security testing.
Ubuntu	A stable Linux distribution for general practice.
Windows	Useful as a target machine or for learning Windows security.
Metasploitable 2	Deliberately vulnerable machine for safe practice.
OWASP Juice Shop	Vulnerable web application for learning web security.

Other Useful Tools

- ✓ Wireshark (network analysis)
- ✓ Nmap (network scanning)
- ✓ Wazuh (SIEM for log monitoring)
- ✓ UFW (Linux firewall)
- ✓ Git (version control)
- ✓ Docker (optional for containers)

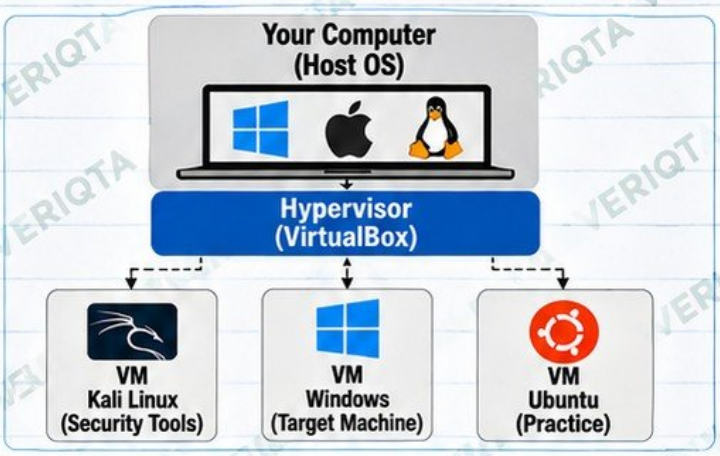
Important Reminders

- ✓ Use only free and legal tools.
- ✓ Download from official websites.
- ✓ Keep your lab isolated.
- ✓ Do not use these tools on real systems without permission.

03 Installing Your Virtualization Platform

1. Hypervisors and Virtual Machines

- A hypervisor is software that lets you run multiple operating systems on one physical computer.
- Each operating system runs in a virtual machine (VM) that is isolated from your main system.
- This allows you to install and test different OS, tools and applications safely.



2. Installing VirtualBox (Example)

1. Go to the official VirtualBox website: https://www.virtualbox.org
2. Download the correct version for your host OS (Windows, Linux or macOS).
3. Install VirtualBox using the default settings.
4. Open VirtualBox and confirm it is working.

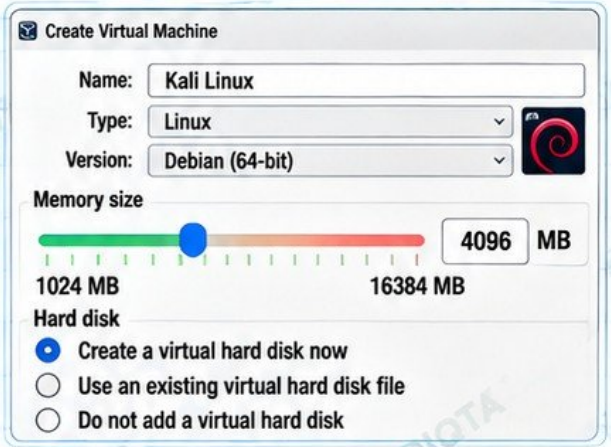


3. Configuring CPU, Memory and Virtual Disk

When you create a new VM, you need to allocate resources from your physical computer.

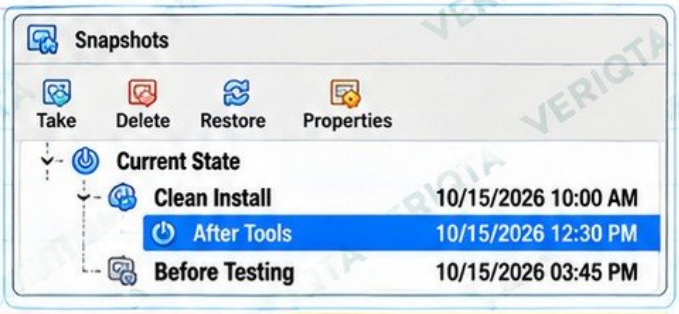
Recommended Settings

Virtual Machine	CPU	RAM	Disk Size
Kali Linux	2–4 cores	4–8 GB	80–100 GB
Windows (Target)	2–4 cores	4–8 GB	60–100 GB
Ubuntu (Practice)	2 cores	2–4 GB	40–60 GB



4. Creating and Managing VM Snapshots

1. A snapshot saves the current state of a VM so you can return to it later.
2. Take a snapshot before installing new tools, updates or making changes.
3. You can create multiple snapshots (e.g. "Clean Install", "After Tools", "Before Testing").
4. To restore, simply select the snapshot and click Restore.



Best Practices

- Allocate enough resources for smooth performance.
- Use snapshots regularly.
- Keep your lab VMs isolated from your main work.
- Give each VM a clear name.

Common Mistakes

- Allocating too little RAM (VM will be slow).
- Not taking snapshots before making changes.
- Storing VMs on a very small disk.
- Connecting lab VMs to your main network by mistake.

04 Building Your First Cybersecurity Lab Network

VERIQTA

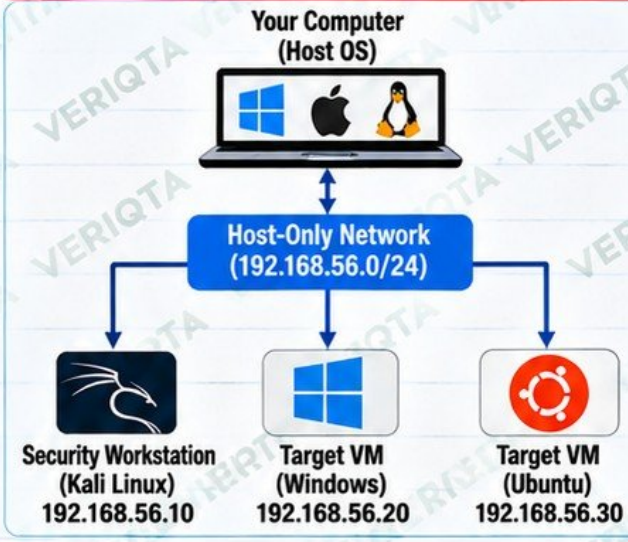
1. Network Types in Virtualization

NAT (Network Address Translation)
- VMs can access the internet.
- Simple and easy to use.
- Good for updates and downloading tools.
- Not fully isolated.

Host-Only Network
- VMs can communicate with the host.
- VMs can communicate with each other.
- No internet access (by default).
- Ideal for a safe lab environment.

Internal Network
- VMs can communicate with each other.
- No access to the host.
- No internet access.
- Useful for simulating isolated environments.

2. Lab Network Design (Example)



3. Creating an Isolated Network in VirtualBox

1. Open VirtualBox and click Tools → Network Manager.
2. Click Create to add a Host-Only Network.
3. Keep the default settings (e.g. 192.168.56.0/24).
4. Make sure "Allow VMs to communicate" is enabled.
5. Click Apply and create your VMs on this network.



4. Example IP Address Plan

Device/VM	IP Address	Purpose
Host (Your PC)	192.168.56.1	Host-only network gateway (for VM communication)
Kali Linux (Security)	192.168.56.10	Security tools and testing
Windows VM (Target)	192.168.56.20	Vulnerable/target machine
Ubuntu VM (Target)	192.168.56.30	Practice and general use

5. Verifying Network Connectivity

1. Check your VM's IP address:
```
ip a     # (Linux)
ipconfig  # (Windows)
```
2. Test VM to VM connectivity:
```
ping 192.168.56.20
```
3. Ensure no internet access (optional for isolation):
```
ping 8.8.8.8   # should fail
```

6. Preventing Accidental Exposure to the Internet

- ☑ Use Host-Only or Internal network for lab VMs.
- ☑ Do not enable internet access unless needed for updates.
- ☑ Keep your lab network separate from your main work.
- ☑ Use snapshots before testing risky tools.
- ☑ Use firewalls inside the VMs.
- ☑ Double-check network settings before starting the lab.
- ☑ Avoid using your personal data in lab VMs.

Key Takeaways

- ✅ NAT gives internet access, but is not fully isolated.
- ✅ Host-Only is best for safe, isolated lab setups.
- ✅ Internal network allows VM-to-VM only.
- ✅ Use private IP addresses (e.g. 192.168.56.0/24).
- ✅ Keep your lab network separate and secure.
- ✅ Always verify network settings.
- ✅ Use snapshots to recover if something goes wrong.

05 Installing Kali Linux Safely

VERIQTA

1. About Kali Linux

- Kali Linux is a Linux distribution designed for security testing, ethical hacking and digital forensics.
- It comes with many preinstalled tools for network analysis, web testing, vulnerabilitty scanning and more.
- You should only use Kali Linux in an isolated lab environment and only for authorized learning and testing.

2. Downloading Kali Linux from Official Sources

1. Go to the official website: https://www.kali.org
2. Click Downloads.
3. Choose the latest version (e.g. Kali Linux 64-bit).
4. Download the ISO file for your platform.



3. Verify the Download (Check Integrity)

1. Download the SHA256 checksum from the official website.
2. Open a terminal or PowerShell.
3. Run the verification command (replace filename with your file name).

Linux / macOS

```
sha256sum kali-linux-*.iso
```

Windows (PowerShell)

```
Get-FileHash kali-linux-*.iso -Algorithm SHA256
```

4. Compare the generated hash with the official hash. They must match.

4. Creating a Kali Linux Virtual Machine

1. Open VirtualBox and click New.
2. Give your VM a name (e.g. Kali Linux).
3. Select Type: Linux and Version: Debian (64-bit).
4. Allocate resources (recommended):

VirtualBox

Resource	Recommended
CPU	2–4 cores
RAM	4–8 GB (minimum 4 GB)
Disk Size	80–100 GB (dynamically allocated)

5. Choose the downloaded Kali Linux ISO file.
6. Complete the setup and start the VM.

5. Update Kali Linux and Test Network Connectivity

1. Start your Kali Linux VM and log in (default user: kali, password: kali).
2. Update the package list:
   ```
   sudo apt update
   ```
3. Upgrade installed packages:
   ```
   sudo apt upgrade -y
   ```
4. Test network connectivity:
   ```
   ping -c 4 8.8.8.8
   # test internet access
   ```
5. Check your IP address:
   ```
   ip a        # (Linux)
   ```



6. Understanding the Lab Environment

- ✓ Kali Linux is a security testing environment.
- ✓ Use it to learn tools, techniques and how attacks and defenses work.
- ✓ Keep it isolated from your personal files and production systems.
- ✓ You can install additional tools as needed.
- ✓ Use snapshots before making major changes so you can easily revert.
- ✓ Practice responsibly and only on systems you own or have permission to test.

Best Practices

- Always download from the official Kali Linux website.
- Verify the ISO file using the SHA256 checksum.
- Use a virtual machine (do not install on your main computer).
- Allocate enough CPU, RAM and disk space.
- Keep your lab network isolated (host-only or internal network).
- Take a snapshot after initial setup.
- Keep Kali Linux updated regularly.

Common Mistakes

- Downloading Kali Linux from unofficial websites.
- Not verifying the ISO file (risk of a modified file).
- Allocating too little RAM or disk space.
- Giving the VM internet access when you want isolation.
- Installing Kali Linux on your main computer.
- Not taking snapshots before making changes.
- Skipping system updates.

06 Linux Commands Every Security Beginner Needs

VERIQTA

1. Navigating Files and Directories

Command	Description	Example	Expected Output
pwd	Show current directory	`pwd`	/home/kali
ls	List files and directories	`ls -la`	Shows files with details
cd	Change directory	`cd /etc`	Moves to /etc
mkdir	Create a directory	`mkdir test`	Creates test directory
rmdir	Remove empty directory	`rmdir test`	Removes test directory

2. Inspecting Processes, Users, and Permissions

Command	Description	Example	Expected Output
ps	Show running processes	`ps aux`	List of processes with details
top	Real-time process monitoring	`top`	Live process view
who	Show logged in users	`who`	List of users
id	Show current user details	`id`	uid=1000(kali) ... groups=...
ls -l	View file permissions	`ls -l file.txt`	-rw-r--r-- 1 kali kali ...
chmod	Change file permissions	`chmod 600 file.txt`	Restrict file access
chown	Change file owner	`chown kali:kali file.txt`	Change owner and group

3. Networking Commands

Command	Description	Example	Expected Output
ip	Show IP address and network info	`ip a`	List of network interfaces and IPs
ss	Show listening ports and connections	`ss -tuln`	List of open ports
ping	Test network connectivity	`ping 8.8.8.8`	Replies from host
curl	Test HTTP/HTTPS connectivity	`curl -I https://kali.org`	HTTP response headers

4. Reading Logs and Checking Services

Command	Description	Example	Expected Output
journalctl	View system logs (systemd)	`journalctl -n 20`	Last 20 log entries
tail	View the end of a log file	`tail -n 20 /var/log/syslog`	Last 20 lines
cat	View log file content	`cat /var/log/auth.log`	Log file content
systemctl	Check service status	`systemctl status ssh`	Service status (active/inactive)
systemctl	Start/stop a service	`sudo systemctl start ssh`	Starts the service

5. Practical Examples (Terminal Output)

Check current directory

```
kali@kali:~$ pwd
/home/kali
```

List files with details

```
kali@kali:~$ ls -la
total 32
drwxr-xr-x 4 kali kali  4096 Oct 15 10:00 .
drwxr-xr-x 3 root root  4096 Oct 15 09:50 ..
-rw-r--r-- 1 kali kali   220 Oct 15 09:55 .bashrc
```

Check IP address

```
kali@kali:~$ ip a
2: eth0: <BROADCAST,MULTICAST,UP>
    inet 192.168.56.10/24  brd 192.168.56.255
    inet6 fe80::a00:27ff:fe4e:66a1/64 scope link
```

View running processes

```
kali@kali:~$ ps aux | head
USER       PID %CPU %MEM    VSZ   RSS TTY
root         1  0.0  0.1 169420  1108 ?
kali      1020  0.1  0.3 734560 24560 ?
kali      1055  0.0  0.1  21724  4320 ?
```

View recent logs

```
kali@kali:~$ journalctl -n 5
Oct 15 09:55:01 kali systemd[1]: Started SSH.
Oct 15 09:55:02 kali sudo[1020]: kali : TTY=pts/0 ;
Oct 15 09:56:10 kali NetworkManager[695]: eth0:
Oct 15 09:56:12 kali systemd[1]: Starting Cleanup.
Oct 15 09:56:12 kali systemd[1]: Finished Cleanup.
```

Check open ports

```
kali@kali:~$ ss -tuln
Netid  State   Recv-Q Send-Q  Local Address:Port
tcp    LISTEN  0      128     0.0.0.0:22
tcp    LISTEN  0      128     127.0.0.1:3306
udp    UNCONN  0      0       0.0.0.0:68
```

6. Best Practices

- ✅ Use only read and safe commands when practicing.
- ✅ Do not delete or modify important system files.
- ✅ Work inside your lab environment (VM).
- ✅ Combine commands with --help or man to learn more.
- ✅ Practice different options (e.g. ls -l, ps aux, journalctl -n 50).
- ✅ Take notes of useful commands and their output.
- ✅ Use tab completion to save time.
- ✅ Always verify what a command does before running it on real systems.

7. Common Mistakes to Avoid

- ❌ Running destructive commands (e.g. `rm -rf`).
- ❌ Modifying system files without understanding.
- ❌ Using commands on your main (non-lab) system.
- ❌ Ignoring command output and error messages.
- ❌ Not checking permissions before accessing files.

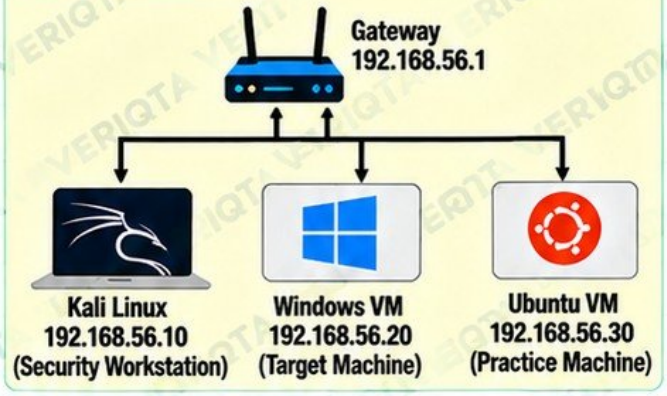
07 Networking Fundamentals for Cybersecurity

VERIQTA

1. IP Addresses, Subnets, Gateways, and Ports

IP Address	A unique address that identifies a device on a network. Example: 192.168.56.10
Subnet	Divides a network into smaller networks. Example: 192.168.56.0/24 (256 addresses)
Gateway	The device that connects your network to other networks. Example: 192.168.56.1
Port	A number that identifies a specific service on a device. Example: 80 (HTTP), 443 (HTTPS)

Example Network

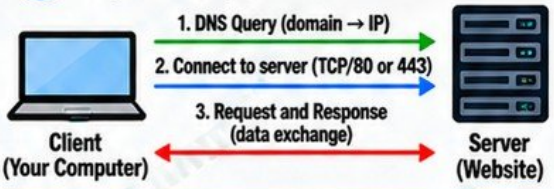


2. Common Network Protocols

Protocol	Port(s)	Purpose	Common Use Cases
TCP	–	Reliable, connection-oriented protocol.	Web, file transfer, email and more.
UDP	–	Faster, connectionless protocol.	DNS, streaming, VoIP, gaming.
DNS	53	Translates domain names to IP addresses.	Finding IP of a website (e.g. kali.org).
HTTP	80	Transfers web data (unencrypted).	Websites (not secure).
HTTPS	443	Transfers web data encrypted with TLS.	Secure websites.

3. How Clients Communicate with Servers

1. You enter a domain name (e.g. www.kali.org).
2. DNS translates the domain to an IP address.
3. Your client (browser) connects to the server IP using a protocol (e.g. TCP).
4. Data is exchanged between client and server.
5. The server responds with the requested content (e.g. a web page).



4. Open Ports and Listening Services

```
kali@kali:~$ ss -tuln
Netid  State   Local Address:Port   Peer Address:Port
tcp    LISTEN  0.0.0.0:22          0.0.0.0:*
tcp    LISTEN  0.0.0.0:80          0.0.0.0:*
tcp    LISTEN  127.0.0.1:3306       0.0.0.0:*
udp    UNCONN  0.0.0.0:53          0.0.0.0:*
```

Key Points

- ✓ An open port means a service is listening for connections.
- ✓ Each service uses a specific port number.
- ✓ Open ports can be scanned with tools like Nmap.
- ✓ Only open ports you need and keep unnecessary services closed.

5. Checking Common Ports with Nmap

```
kali@kali:~$ nmap -sT 192.168.56.20
Starting Nmap 7.94 ( https://nmap.org )
Nmap scan report for 192.168.56.20
Host is up (0.0009ss latency).
PORT       STATE      SERVICE
22/tcp     open       ssh
80/tcp     open       http
3306/tcp   open       mysql
Nmap done: 1 IP address (1 host up)
```

6. Troubleshooting Basic Connectivity Failures

Problem	Possible Cause	How to Fix
Cannot ping another VM	• Wrong IP address • Firewall blocking • Network not connected	• Check IP with `ip a` • Allow ICMP or disable firewall (lab) • Verify VM network settings
DNS not working	• No internet access • Wrong DNS server	• Check gateway and internet • Use 8.8.8.8 or your lab DNS
Cannot access a website	• Service not running • Firewall blocking port • Wrong port	• Check with `ss -tuln` • Allow the port (e.g. 80/443) • Verify correct URL and port
Connection times out	• Host is down • Network misconfiguration • Service not listening	• Check VM is running • Verify network settings • Check service status

7. Useful Commands Summary

Command	Purpose	Example
ip a	Show IP addresses	`ip a`
ip route	Show routing table	`ip route`
ping	Test connectivity	`ping 8.8.8.8`
ss -tuln	Show open ports	`ss -tuln`
nslookup	Query DNS	`nslookup kali.org`
curl	Test HTTP/HTTPS	`curl -I https://kali.org`
traceroute	Trace network path	`traceroute kali.org`
journalctl	View system logs	`journalctl -n 20`
systemctl	Check service status	`systemctl status ssh`

08 Network Discovery and Port Scanning with Nmap

1. What is Network Discovery and Port Scanning?

- Network discovery finds live hosts (devices) on a network.
- Port scanning checks which ports are open, closed or filtered on a host.
- It helps you understand what services are running.
- You should only scan machines you own or have permission to test.
- It is a key skill for security assessment and troubleshooting.

2. Installing Nmap on Kali Linux

1. Nmap is preinstalled on Kali Linux.
2. Check the version:
   ```
   nmap --version
   ```
3. If not installed, run:
   ```
   sudo apt update
   sudo apt install nmap
   ```
4. Verify the installation:
   ```
   nmap --version
   ```

```
kali@kali:~$ nmap --version
Nmap version 7.94 ( https://nmap.org )
Platform: x86_64-pc-linux-gnu
Compiled with: .....
NSE: Loaded 159 scripts for scanning.
```

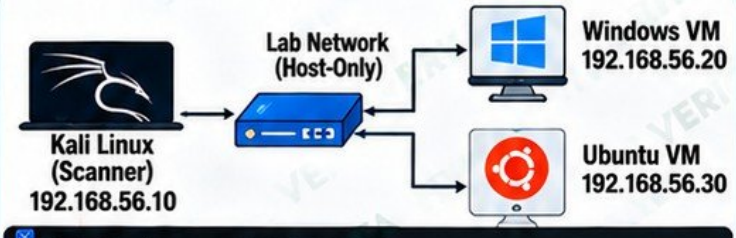
3. Basic Nmap Scans

Scan Type	Command	Purpose	Example Output (Shortened)
Ping Scan	`nmap -sn 192.168.56.0/24`	Find live hosts on the network	Nmap scan report for 192.168.56.10 Nmap scan report for 192.168.56.20
Basic Port Scan	`nmap 192.168.56.10`	Check common open ports	22/tcp open ssh 80/tcp open http
Service Detection	`nmap -sV 192.168.56.10`	Identify services and versions	22/tcp open ssh OpenSSH 9.2p1 80/tcp open http Apache 2.4.58
OS Detection	`nmap -O 192.168.56.10`	Guess the operating system	Running: Linux OS details: Linux 5.x
Detailed Scan	`nmap -sS -sV -O 192.168.56.10`	Comprehensive scan (TCP SYN)	Shows open ports, services, OS and more.

4. Understanding Port States

Port State	Meaning	Common Reasons	Example
Open	A service is listening on the port.	Application or service running and accepting connections.	22/tcp open ssh
Closed	No service is listening on the port.	Port is reachable but no application is listening.	80/tcp closed
Filtered	Nmap cannot determine if the port is open or closed.	Firewall or network device is blocking or filtering the traffic.	443/tcp filtered

5. Scanning an Authorized Lab Network (Example)




```
kali@kali:~$ nmap 192.168.56.0/24
Starting Nmap 7.94 ( https://nmap.org ) at 2026-10-15 10:00
Nmap scan report for 192.168.56.10
Nmap scan report for 192.168.56.20
Nmap scan report for 192.168.56.30
Nmap done: 256 IP addresses (3 hosts up) scanned in 12.34 seconds
```

6. Identifying Services and Documenting Results

1. Run a service detection scan:
   ```
   nmap -sV 192.168.56.20
   ```
2. Save the results to a file:
   ```
   nmap -sV 192.168.56.20 -oN scan-results.txt
   ```
3. View the results:
   ```
   cat scan-results.txt
   ```
4. Document open ports, services and versions for your lab report.

Best Practices

- Only scan machines you own or have permission to test.
- Start with a simple scan and then use more detailed options.
- Combine Nmap with other tools for deeper analysis.
- Save your scan results for future reference.
- Be careful and avoid aggressive scans on networks you do not own.

Common Mistakes

- Scanning external networks without permission.
- Using aggressive scans that may trigger security alerts.
- Not interpreting filtered ports correctly.
- Ignoring service version information.
- Not saving or documenting scan results.
- Assuming all open ports are vulnerable.

09 Capturing Network Traffic with Wireshark

VERIQTA

1. What are Packets and Packet Capture?

- A packet is a small unit of data sent over a network.
- Packet capture records and shows these packets in real time.
- It helps you see what devices are communicating and what data is being sent.
- Wireshark is a tool used to capture, view and analyze network traffic.



Example of a Network Packet

Ethernet Header	IP Header	TCP Header	Data (Payload)

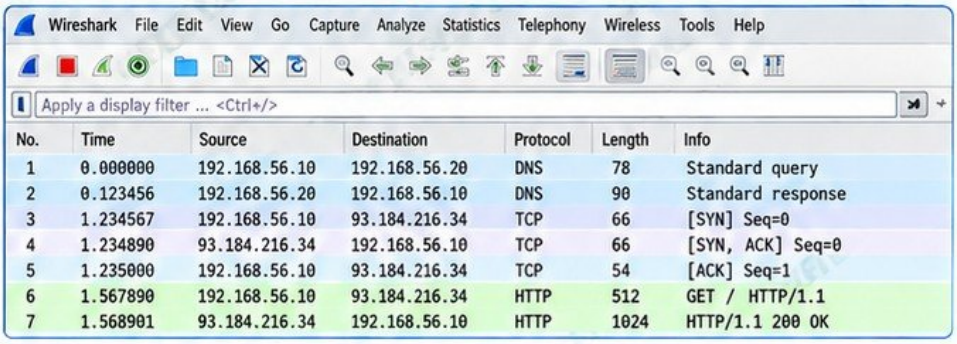
2. Installing Wireshark

1. On Kali Linux (preinstalled): Check the version:
   ```
   wireshark --version
   ```
2. If not installed, run:
   ```
   sudo apt update
   sudo apt install wireshark
   ```
3. Start Wireshark:
   ```
   wireshark
   ```
4. Allow non-root user to capture packets (if needed):
   ```
   sudo usermod -aG wireshark $USER
   ```

Wireshark

3. Capturing Traffic from Your Lab Network

1. Open Wireshark.
2. Select your network interface (e.g. eth0 or enp0s3).
3. Click the Start button (shark fin icon).
4. Let it capture traffic while you perform some activities (e.g. open a website, ping a host).
5. Click Stop when done.



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.56.10	192.168.56.20	DNS	78	Standard query
2	0.123456	192.168.56.20	192.168.56.10	DNS	90	Standard response
3	1.234567	192.168.56.10	93.184.216.34	TCP	66	[SYN] Seq=0
4	1.234890	93.184.216.34	192.168.56.10	TCP	66	[SYN, ACK] Seq=0
5	1.235000	192.168.56.10	93.184.216.34	TCP	54	[ACK] Seq=1
6	1.567890	192.168.56.10	93.184.216.34	HTTP	512	GET / HTTP/1.1
7	1.568901	93.184.216.34	192.168.56.10	HTTP	1024	HTTP/1.1 200 OK

4. Filtering Common Traffic Types

Traffic Type	Filter Expression	Description	Example
All Traffic	(no filter)	Shows all captured packets	-
DNS	dns	Show only DNS traffic	No. Time Source Destination Protocol Info 12 1.234567 192.168.56.10 8.8.8.8 DNS Standard query
TCP	tcp	Show only TCP traffic	No. Time Source Destination Protocol Info 25 2.345678 192.168.56.10 93.184.216.34 TCP 66 [SYN]
HTTP	http	Show only HTTP traffic (unencrypted)	No. Time Source Destination Protocol Info 42 3.456789 192.168.56.10 93.184.216.34 HTTP GET /index.html
HTTPS	tls	Show only HTTPS (encrypted) traffic	58 4.567890 192.168.56.10 142.250.72.14 TLS Client Hello

5. Recognizing Normal Network Communication

Activity	What You See	Example
DNS Lookup	DNS query and response packets	Your machine requests a domain name (e.g. www.google.com)
Web Browsing (HTTP)	TCP connection and HTTP GET/response	GET / HTTP/1.1 and 200 OK
Web Browsing (HTTPS)	TLS handshake and encrypted traffic	Client Hello and Server Hello
System Updates	Multiple connections to known servers	Connections to official update servers (e.g. packages.kali.org)
Internal Communication	Traffic between your lab machines	e.g. 192.168.56.10 ↔ 192.168.56.20

6. Practical Example: Capture and Analyze DNS Traffic

1. Start a capture on your network interface.
2. Open a website (e.g. www.kali.org).
3. Apply the filter: `dns`
4. Look at the DNS query and response packets.

No.	Time	Source	Destination	Protocol	Info
12	1.234567	192.168.56.10	8.8.8.8	DNS	Standard query A kali.org
13	1.345678	8.8.8.8	192.168.56.10	DNS	Standard response A 104.26.3.1
14	1.456789	192.168.56.10	8.8.8.8	DNS	Standard query AAAA kali.org
15	1.567890	8.8.8.8	192.168.56.10	DNS	Standard response No such name

Best Practices

- Only capture traffic on networks you own or have permission to test.
- Use display filters to focus on specific traffic.
- Stop the capture when you are done.
- Save packet captures for later analysis (.pcap file).
- Be mindful of sensitive information in captured data.

Common Mistakes

- Capturing traffic on networks without permission.
- Not using filters (too much irrelevant data).
- Capturing for too long (large files and performance issues).
- Misinterpreting normal traffic as suspicious.
- Sharing packet captures that contain sensitive information.

10 Building a Vulnerable Practice Machine

VERIQTA

1. Understanding Deliberately Vulnerable Applications

- Deliberately vulnerable applications are programs designed with known security flaws for learning and practice.
- They help you understand how real-world attacks work in a safe and legal environment.
- OWASP Juice Shop is a popular vulnerable web application created by the OWASP Foundation.
- It simulates real security weaknesses such as broken authentication, injection flaws and insecure configurations.
- You should only use vulnerable applications in an isolated lab environment and never on public networks.



OWASP Juice Shop

A deliberately vulnerable web application for security training.

2. Deploying OWASP Juice Shop in an Isolated Environment

1. Update your system and install Docker (if not already installed).

```
sudo apt update
sudo apt install docker.io -y
```

2. Pull the Juice Shop image.

```
sudo docker pull bkimminich/juice-shop
```

3. Run the container.

```
sudo docker run -d -p 3000:3000 \
--name juice-shop bkimminich/juice-shop
```

4. Verify it is running.

```
sudo docker ps
```

CONTAINER ID	IMAGE	PORTS	NAMES
a1b2c3d4	bkimminich/juice-shop	0.0.0.0:3000->3000/tcp	juice-shop

3. Accessing Juice Shop in Your Lab

1. Open your browser and go to:
   `http://localhost:3000`
   (if on the same machine)
2. Or use your VM's IP address:
   `http://192.168.56.20:3000`
   (if accessing from another VM in your isolated network)
3. You should see the OWASP Juice Shop home page.
4. Do not expose this application to the internet.

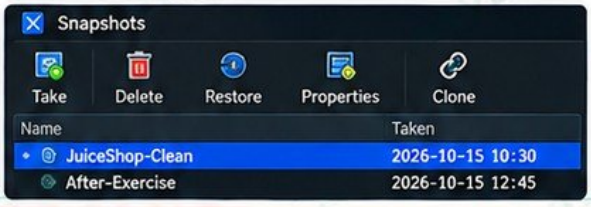


4. Understanding Application Security Weaknesses

Weakness Type	Description	Example in Juice Shop
Broken Authentication	Weak or missing authentication controls.	Login with default or manipulated credentials.
Injection Flaws	Unvalidated user input allows malicious data.	SQL injection in search function.
Sensitive Data Exposure	Leaking sensitive information.	View hidden data or configuration files.
Broken Access Control	Accessing restricted resources.	Access admin functions without proper rights.
Insecure Configuration	Poor security settings.	Exposed endpoints and verbose error messages.

5. Using Snapshots to Restore the Lab

1. After setting up Juice Shop, create a VM snapshot (e.g. "JuiceShop-Clean").
2. If you make changes or break the environment, restore the snapshot instead of reinstalling.
3. This allows you to safely repeat exercises over and over.



Best Practices

- Keep Juice Shop in an isolated lab network.
- Use Docker or a separate VM.
- Do not expose it to the internet.
- Take a snapshot after a clean installation.
- Use it for learning and practice only.
- Reset using snapshots when needed.

Common Mistakes

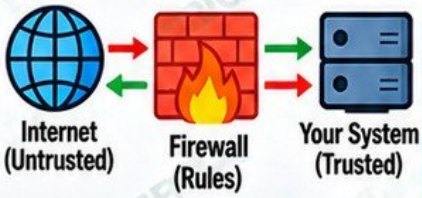
- Exposing Juice Shop to the public internet.
- Using it on a production or real network.
- Not taking a snapshot before making changes.
- Skipping basic security checks.
- Forgetting to restore the lab after breaking it.

11 Understanding Firewalls and Network Defense

VERIQTA

1. What Firewalls Protect and How Rules Work

- A firewall controls network traffic based on a set of rules.
- It allows legitimate traffic and blocks unwanted or dangerous traffic.
- Firewalls help protect your systems from unauthorized access, attacks and scanning.
- Rules define what traffic is allowed or denied based on IP addresses, ports and protocols.
- In a lab environment, you use firewalls to practice and understand how network security works.



→ Allowed traffic (matches rule)
→ Blocked traffic (does not match rule)

2. Inbound and Outbound Traffic

Inbound Traffic

Traffic coming from external networks to your system.

Example: Someone trying to connect to your SSH or web server.

Outbound Traffic

Traffic going from your system to external networks.

Example: Your system downloading updates or accessing a website.

3. Linux Firewall Concepts Using UFW

- UFW (Uncomplicated Firewall) is a simple tool to manage firewall rules on Ubuntu and Kali Linux.
- It controls incoming and outgoing traffic.
- Rules are based on ports, protocols and IP addresses.
- UFW is easy to use and good for learning firewall basics.

Install and check UFW

```
kali@kali:~$ sudo apt update
kali@kali:~$ sudo apt install ufw -y
kali@kali:~$ sudo ufw status
Status: inactive
```



4. Allowing and Blocking Controlled Lab Connections

Task	Command	Description
Allow SSH (port 22)	`sudo ufw allow 22/tcp`	Allow inbound SSH connections.
Allow a specific IP	`sudo ufw allow from 192.168.56.10 to any port 22`	Allow SSH from a specific IP address.
Allow web traffic	`sudo ufw allow 80,443/tcp`	Allow HTTP and HTTPS.
Block a port	`sudo ufw deny 23/tcp`	Block Telnet (port 23).
Delete a rule	`sudo ufw delete allow 22/tcp`	Remove a specific rule.
Enable UFW	`sudo ufw enable`	Turn on the firewall.
Check status	`sudo ufw status verbose`	View all rules and details.

5. Verifying Rules and Testing Connections

1. Check the firewall status:
   `sudo ufw status verbose`
2. Test a connection from another machine (in your lab):
   `ssh 192.168.56.20`
3. Check if a port is open:
   `sudo nmap -sT 192.168.56.20`
4. Check blocked ports (should be filtered):
   `sudo nmap -sT -p 23 192.168.56.20`

6. Example: UFW Rule Output

```
kali@kali:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW       Anywhere
80/tcp                     ALLOW       Anywhere
443/tcp                    ALLOW       Anywhere
23/tcp                     DENY        Anywhere
```

7. Best Practices

- ✓ Start with a simple set of rules.
- ✓ Allow only the ports you need.
- ✓ Use specific IP addresses when possible.
- ✓ Test your rules after making changes.
- ✓ Keep an alternative way to access your machine (e.g. a VM console).
- ✓ Document your firewall rules.
- ✓ Be careful not to lock yourself out.
- ✓ Disable or reset UFW if you lose access: sudo ufw disable

8. Common Mistakes to Avoid

- ✗ Allowing unnecessary ports.
- ✗ Using allow from any when not needed.
- ✗ Blocking your own access (e.g. SSH).
- ✗ Not testing rules after changes.
- ✗ Forgetting outbound traffic needs.
- ✗ Not documenting firewall rules.
- ✗ Disabling the firewall completely instead of using proper rules.
- ✗ Making changes without a plan to recover access.

12 Linux Logs and Security Event Investigation

VERIQTA

1. Understanding Linux Logs

- Logs are records of events that happen on your system.
- They help you understand what users do, what services do, and what errors occur.
- Key log types for security investigation:

Log Type	Purpose	Common Location
Authentication Logs	Login attempts (success and failed)	/var/log/auth.log (or /var/log/secure)
System Logs	System events, service status, kernel	/var/log/syslog (or /var/log/messages)
Application Logs	Application-specific events (e.g. web server)	/var/log/apache2/ /var/log/nginx/
Journal Logs	Centralized systemd logs	View with journalctl

2. Using journalctl and /var/log

View recent system logs with journalctl

```
kali@kali:~$ sudo journalctl -xe
Oct 15 10:21:32 kali systemd[1]: Starting ssh.service...
Oct 15 10:21:33 kali sshd[1234]: Server listening on 0.0.0.0 port 22.
Oct 15 10:21:33 kali systemd[1]: Started ssh.service.
```

View authentication logs

```
kali@kali:~$ sudo cat /var/log/auth.log | tail -10
Oct 15 10:25:12 kali sshd[3456]: Failed password for invalid user
    admin from 192.168.56.10 port 52314 ssh2
Oct 15 10:25:14 kali sshd[3456]: Accepted password for kali
    from 192.168.56.20 port 52320 ssh2
```

Search for specific keywords

```
kali@kali:~$ sudo grep "Failed password" /var/log/auth.log
Oct 15 10:25:12 kali sshd[3456]: Failed password for invalid user
    admin from 192.168.56.10 port 52314 ssh2
Oct 15 10:27:03 kali sshd[3521]: Failed password for root from
    192.168.56.30 port 49872 ssh2
```

3. Recognizing Failed Login Attempts

- Failed login attempts can indicate brute force attacks or unauthorized access.
- Check authentication logs for repeated failed attempts.
- Look for the source IP address, username and timestamp.
- Multiple failed attempts from the same IP may be suspicious.

```
kali@kali:~$ sudo grep "Failed password" /var/log/auth.log
Oct 15 10:25:12 kali sshd[3456]: Failed password for invalid user admin from 192.168.56.10 port 52314 ssh2
Oct 15 10:26:01 kali sshd[3480]: Failed password for root from 192.168.56.10 port 52345 ssh2
Oct 15 10:27:03 kali sshd[3521]: Failed password for kali from 192.168.56.30 port 49872 ssh2
```

4. Investigating Suspicious Events Using Timestamps

1. Identify the time of the event. Example: Oct 15 10:25:12
2. Check related logs around the same time. Use journalctl or /var/log files.
3. Look for patterns:
   - multiple failed logins
   - unexpected users
   - unusual IP addresses
   - service restarts
4. Correlate events across logs to understand what happened.



5. Simple Evidence-Based Investigation Workflow

1. Identify the alert or suspicious event (e.g. multiple failed logins).
2. Gather evidence from logs (journalctl, /var/log).
3. Verify timestamps and related events.
4. Identify the source IP, user and activity.
5. Determine the impact (successful login, service change, etc.).
6. Document your findings with commands and log output.



6. Useful Log Investigation Commands

Command	Purpose
sudo journalctl -xe	View recent system logs (detailed).
sudo journalctl -u ssh	View logs for a specific service (SSH).
sudo cat /var/log/auth.log	View authentication logs.
sudo grep "Failed password" /var/log/auth.log	Find failed login attempts.
sudo tail -f /var/log/syslog	Watch logs in real time.
sudo journalctl --since "1 hour ago"	View logs from a specific time.
sudo grep <IP_ADDRESS> /var/log/auth.log	Find activity from a specific IP.

7. Best Practices

- Regularly review authentication and system logs.
- Use specific time ranges when investigating.
- Look for unusual patterns or repeated failures.
- Keep log evidence when documenting an incident.
- Combine multiple log sources for a complete picture.
- Practice in your lab to build real investigation skills.

8. Common Mistakes to Avoid

- Ignoring log warnings and failed login attempts.
- Not checking timestamps and related events.
- Only looking at one log file.
- Not verifying the source IP address.
- Deleting or overwriting logs too early.
- Making conclusions without enough evidence.

13 Building a Basic Security Monitoring Lab

VERIQTA

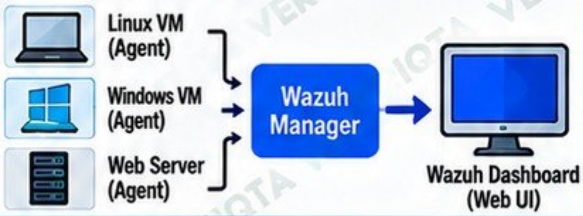
1. What is SIEM?

- A Security Information and Event Management (SIEM) system collects, correlates and analyzes logs from multiple systems in a centralized platform.
- It helps you detect suspicious activity, security threats and incidents.
- SIEM gives you better visibility, faster detection and easier investigation.
- Examples of SIEM tools include Wazuh, Splunk and ELK Stack.



2. Understanding Log Collection and Centralized Monitoring

- Logs are collected from servers, workstations, network devices and applications.
- Logs are sent to a central server for storage, analysis and alerting.
- Centralized monitoring helps you see the big picture across all your systems.
- It makes it easier to detect unusual activity and respond quickly.



3. Introducing Wazuh and Its Main Components

- Wazuh is an open-source SIEM platform for threat detection, security monitoring and compliance.
- It collects logs, analyzes them and generates alerts for suspicious activity.
- Main components:

wazuh

Component	Description
Wazuh Agent	Installed on endpoints (servers, workstations) to collect logs and send them to the manager.
Wazuh Manager	Receives logs, analyzes events, generates alerts and stores data (uses OpenSearch).
OpenSearch	Stores and indexes logs for search and visualization.
Wazuh Dashboard	Web interface to view alerts, logs and reports.

4. Planning a Small Monitoring Environment

1. Set up a lab network (host-only or internal network).
2. Deploy a Wazuh manager (use official installation guide or a prebuilt VM).
3. Install Wazuh agents on target machines (e.g. Kali Linux and a vulnerable machine).
4. Configure the agents to connect to the manager.
5. Verify that logs are being received in the dashboard.

```
kali@kali:~$ # Install Wazuh agent on Ubuntu/Kali
sudo apt update
sudo apt install wazuh-agent -y
# Configure manager IP
sudo nano /var/ossec/etc/ossec.conf
# Start agent
sudo systemctl start wazuh-agent
sudo systemctl status wazuh-agent
```

5. Understanding Alerts and Security Events

- Alerts are generated when suspicious or unusual activity is detected in the logs.
- Examples of security events:

Failed login attempts	Multiple failed SSH logins
Privilege escalation	User running sudo commands
Malware detection	Detection of malicious files
System changes	Important configuration changes
Suspicious network activity	Unusual connections or scans



6. Best Practices

- Use a dedicated isolated lab network.
- Monitor both Linux and Windows machines.
- Collect and analyze different types of logs.
- Regularly review alerts and understand what they mean.
- Keep your lab updated and take snapshots before making changes.
- Document your setup and findings.

7. Common Mistakes to Avoid

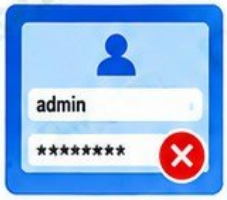
- Not installing agents on target machines.
- Not checking if logs are actually being received.
- Ignoring important alerts.
- Using weak or default configurations.
- Not securing the Wazuh dashboard.
- Running the lab on a public network.
- Not taking snapshots before making changes.

14 Detecting Suspicious Login Activity

VERIQTA

1. Understanding Authentication Failures

- Authentication failures happen when someone tries to log in with the wrong username, password or key.
- They can indicate brute force attacks, credential stuffing or unauthorized access attempts.
- Authentication logs help you see who tried to log in, when it happened and from which IP address.
- Not all failed logins are attacks. Some are legitimate mistakes.



Login Failed
Invalid username or password

2. Generating Controlled Failed Login Events

1. On a lab machine, try logging in with a wrong password a few times.
   `ssh user@localhost`
2. Enter an incorrect password multiple times.
3. You can also try an invalid username.
   `ssh invaliduser@localhost`
4. Do this only in your lab environment and log a small number of attempts (e.g. 5–10).



⚠ Do this only on machines you own or have explicit permission to test.

3. Finding Evidence in Authentication Logs

On most Linux systems, authentication logs are in:

System	Log File / Command
Ubuntu / Debian	/var/log/auth.log
Kali Linux	/var/log/auth.log
RHEL / CentOS	/var/log/secure
View with journalctl	sudo journalctl -u ssh
View failed logins only	sudo grep "Failed password" /var/log/auth.log
Follow logs in real time	sudo tail -f /var/log/auth.log

4. Example: Failed Login Attempts in Logs

```
kali@kali:~$ sudo grep "Failed password" /var/log/auth.log
Oct 15 10:12:01 kali sshd[1234]: Failed password for
      invalid user admin from 192.168.56.10 port 54322 ssh2
Oct 15 10:12:05 kali sshd[1234]: Failed password for
      invalid user admin from 192.168.56.10 port 54324 ssh2
Oct 15 10:12:09 kali sshd[1234]: Failed password for
      kali from 192.168.56.10 port 54326 ssh2
Oct 15 10:12:13 kali sshd[1234]: Failed password for
      kali from 192.168.56.10 port 54328 ssh2
Oct 15 10:12:17 kali sshd[1234]: Failed password for
      kali from 192.168.56.10 port 54330 ssh2
```

5. Identifying Repeated Failures and Unusual Patterns

- Multiple failed attempts from the same IP address in a short time can indicate a brute force attack.
- Look for repeated failed logins for the same user or multiple users.
- Check if the attempts come from unusual or external IP addresses.
- Use commands to summarize activity.

```
# Count failed logins by IP address
sudo grep "Failed password" /var/log/auth.log | \
awk '{print $(NF-3)}' | sort | uniq -c | sort -nr
  25 192.168.56.10
   8 203.0.113.45
   3 198.51.100.23
```

6. Distinguishing Suspicious Activity from Normal Mistakes

Factor	Likely Normal	Likely Suspicious
Number of failed attempts	1 – 3	Many (e.g. 10+)
Time period	Spread out over time	Rapid and repeated in a short time
Source IP address	Your own network	Unknown or external IP
User account	Your usual account	Multiple accounts or non-existent users
Login time	During normal working hours	Late at night or unusual times
Pattern	Occasional mistakes	Repeated and consistent failures

7. Best Practices

- Regularly review authentication logs.
- Look for repeated failed login attempts.
- Monitor for unusual IP addresses or patterns.
- Set up alerts in your monitoring tool (e.g. Wazuh).
- Keep records of your findings.
- Use strong passwords and key-based authentication.

8. Common Mistakes to Avoid

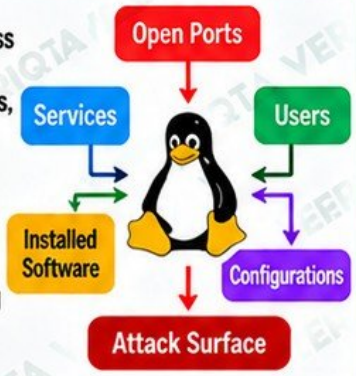
- Ignoring failed login attempts.
- Assuming all failed logins are attacks.
- Not checking the source IP address.
- Not looking at the time and frequency.
- Only looking at one log file.
- Not taking action when you see clear suspicious activity in your lab or production environment.

15 Linux System Hardening

VERIQTA

1. Understanding Attack Surfaces

- An attack surface is everything that can be used to gain unauthorized access to your system.
- It includes open ports, running services, installed software, user accounts and misconfigurations.
- The smaller the attack surface, the harder it is for attackers to find and exploit weaknesses.
- Regularly review and remove anything you do not need.



2. Removing Unnecessary Services

1. List all running services:
```
systemctl list-units --type=service --state=running
```
2. Identify services you do not need.
3. Stop and disable unnecessary services:
```
sudo systemctl stop service_name
sudo systemctl disable service_name
```
4. Verify the service is stopped:
```
systemctl status service_name
```

3. Configuring Secure User Permissions

- Use strong passwords and avoid shared accounts.
- Create non-root users for daily work.
- Give the minimum permissions needed (least privilege).
- Use groups to manage access.
- Check file and directory permissions regularly.

Task	Command
Create a new user	`sudo adduser devuser`
Add user to sudo group	`sudo usermod -aG sudo devuser`
Check user permissions	`ls -l /home`
Change file permissions	`chmod 750 script.sh`
Change ownership	`sudo chown user:group file.txt`

4. Protecting SSH Access

1. Change the default SSH port (optional).
2. Disable root login.
3. Use key-based authentication.
4. Limit access to specific users or IP addresses.
5. Keep SSH configuration simple and secure.

`/etc/ssh/sshd_config` (key settings)
```
Port 2222
PermitRootLogin no
PasswordAuthentication no
AllowUsers devuser
# Allow from specific IP (optional)
AllowUsers devuser@192.168.56.0/24
```

5. Applying Security Updates

1. Regularly install security updates to fix known vulnerabilities.
2. Check for available updates:
```
sudo apt update
```
3. Install updates:
```
sudo apt upgrade -y
```
4. Remove unused packages:
```
sudo apt autoremove -y
```

6. Verifying Hardening Changes

1. Check running services:
```
systemctl list-units --type=service --state=running
```
2. Verify open ports:
```
sudo ss -tulnp
```
3. Check user accounts:
```
cat /etc/passwd | cut -d: -f1
```
4. Confirm SSH settings:
```
sudo sshd -T | grep -E "port|permitrootlogin|passwordauthentication"
```
4. Review system security:
```
sudo lynis audit system
```

7. Best Practices

- Keep your system minimal and remove what you do not use.
- Use non-root accounts and strong passwords.
- Keep SSH access secure.
- Apply security updates regularly.
- Review logs for suspicious activity.
- Verify your hardening changes.
- Document the changes you make.

8. Common Mistakes to Avoid

- Leaving unnecessary services running.
- Keeping default SSH settings.
- Using weak or shared passwords.
- Giving more permissions than needed.
- Forgetting to apply security updates.
- Not verifying that hardening changes actually work.
- Locking yourself out by applying incorrect SSH rules.

16 Vulnerability Scanning and Risk Prioritization

1. Understanding Vulnerabilities, Exposures, and Risk

- A vulnerability is a weakness that can be exploited.
- An exposure is when the vulnerability is accessible (e.g. open to a network).
- Risk is the potential impact if the vulnerability is exploited.
- Not all vulnerabilities are high risk. Risk depends on exploitability, impact and your environment.



2. Introducing Vulnerability Scanners

- Vulnerability scanners automatically check systems for known vulnerabilities.
- They compare your system against a database of known issues (e.g. CVEs).
- Popular tools: Nessus, OpenVAS (Greenbone), Nmap (basic), and others.
- Scanners help you find and fix issues before attackers do.



3. Running Authorized Scans Against Lab Targets

1. Make sure you have permission to scan (only scan your own lab machines).
2. Identify the target IP address or network.
3. Choose a scanner (e.g. OpenVAS or Nessus).
4. Run a basic scan and wait for it to complete.
5. Review the scan summary.



4. Example: Running a Basic Scan with OpenVAS

```
kali@kali:~$ sudo gvm-start
kali@kali:~$ openvas-start
kali@kali:~$ sudo gvm-cli socket --gmp "create target
name=Lab-Server hosts=192.168.56.20"
kali@kali:~$ sudo gvm-cli socket --gmp "create task
target=Lab-Server name=Lab-Scan"
kali@kali:~$ sudo gvm-cli socket --gmp "start task
<task-id>"
```

5. Reading Vulnerability Findings

Scan results usually show:

- Vulnerability name (e.g. OpenSSH outdated version)
- Severity (Critical, High, Medium, Low)
- Description (what the issue is)
- Affected component (service or package)
- Recommendation (how to fix it)

Severity	CVE	Vulnerability	Affected Service
Critical	CVE-2024-1234	OpenSSH Vulnerability	ssh (port 22)
High	CVE-2023-5678	Outdated Apache Version	http (port 80)
Medium	CVE-2022-9012	OpenSSL Vulnerability	ssl/tls
Low	CVE-2021-3456	Information Disclosure	http (port 80)

6. Prioritizing Fixes and Verifying Remediation

1. Focus on high and critical vulnerabilities first.
2. Check if the vulnerability is actually exploitable in your environment.
3. Apply the recommended fix (e.g. update a package or change a configuration).
4. Run the scan again to verify that the issue is resolved.
5. Document what was fixed and the new results.



Vulnerability	Severity	Status (Before)	Status (After)
OpenSSH outdated	High	Open	Fixed
Apache version	Medium	Open	Fixed
SSL/TLS issue	High	Open	Fixed

7. Understanding False Positives

- A false positive is when the scanner reports a vulnerability that is not actually a real issue.
- It can happen due to:
  - Incorrect detection by the scanner.
  - Missing context (e.g. service not actually in use).
  - Custom configurations that are secure.
- Always verify findings manually before taking action.
- Not every finding needs to be fixed.



8. Best Practices

- ✓ Scan only authorized systems.
- ✓ Keep your scanner's vulnerability database updated.
- ✓ Regularly scan your systems.
- ✓ Focus on high and critical risks first.
- ✓ Verify and document your findings.
- ✓ Retest after applying fixes.
- ✓ Understand the context and validate false positives.
- ✓ Combine scanning with manual review.

17 Web Application Security Fundamentals

VERIQTA

1. Understanding Common Web Application Weaknesses

- Web applications can have security weaknesses that attackers can exploit.
- Common weaknesses include injection attacks, broken authentication, insecure configuration, and sensitive data exposure.
- These weaknesses can lead to data theft, account takeover or system compromise.
- Understanding how they work helps you build and maintain more secure applications.



2. Introducing the OWASP Top 10

- The OWASP Top 10 is a list of the most critical web application security risks.
- It helps developers and security engineers focus on the most important issues.
- The list is updated regularly to reflect current threats.

OWASP Top 10 (2021)		Description
A01	Broken Access Control	Unauthorized access to resources
A02	Cryptographic Failures	Sensitive data exposure
A03	Injection	Executing malicious input
A04	Insecure Design	Security design flaws
A05	Security Misconfiguration	Incorrect or default settings
A06	Vulnerable Components	Using outdated software
A07	Identification and Authentication Failures	Weak authentication
A08	Software and Data Integrity Failures	Untrusted code or data
A09	Security Logging and Monitoring Failures	Insufficient logging and detection
A10	Server-Side Request Forgery (SSRF)	Abuse of server-side requests

3. Authentication, Authorization, and Input Validation

Authentication

Verifies who you are (e.g. username and password).



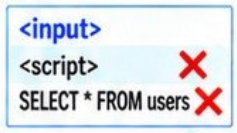
Authorization

Determines what you are allowed to do after authentication (e.g. admin or regular user).



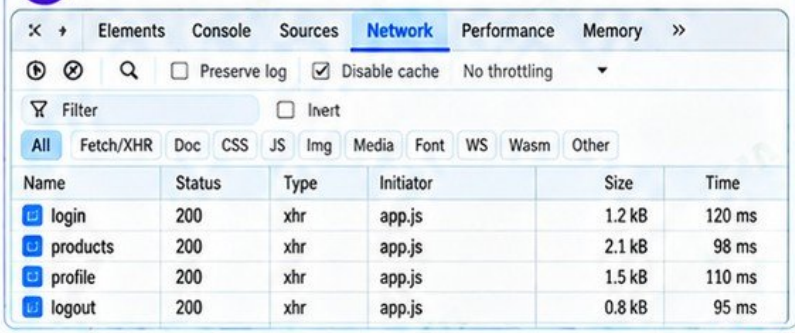
Input Validation

Checks and cleans user input to prevent malicious data (e.g. SQL injection, XSS).



4. Inspecting Browser Requests with Developer Tools

1. Open your browser and go to the web application.
2. Press F12 to open Developer Tools.
3. Go to the Network tab to see HTTP requests and responses.
4. Click on a request to view headers, parameters and response data.
5. Look for interesting information such as tokens, IDs and input values.



Name	Status	Type	Initiator	Size	Time
login	200	xhr	app.js	1.2 kB	120 ms
products	200	xhr	app.js	2.1 kB	98 ms
profile	200	xhr	app.js	1.5 kB	110 ms
logout	200	xhr	app.js	0.8 kB	95 ms

5. Practicing Safe Security Testing in Juice Shop

- OWASP Juice Shop is a deliberately vulnerable web application for learning and practice.
- You can safely test and explore security weaknesses in a controlled environment.
- Try finding common issues such as:
  - Broken authentication
  - Input validation issues
  - Sensitive data exposure
  - Insecure direct object references
  - Cross-site scripting (XSS)




```
# Start Juice Shop using Docker
docker run -d --name juice-shop -p 3000:3000
bkimminich/juice-shop
# Open in browser
http://localhost:3000
# Explore and complete challenges safely
```

6. Best Practices

- Understand the OWASP Top 10.
- Use strong authentication and authorization.
- Always validate and sanitize user input.
- Keep software and dependencies updated.
- Use developer tools to understand how the application works.
- Practice only in authorized and controlled environments (e.g. Juice Shop).
- Document what you find and how to fix it.

7. Common Mistakes to Avoid

- Testing without permission.
- Using security testing techniques on live production systems.
- Ignoring the OWASP Top 10.
- Trusting user input.
- Exposing sensitive data in requests or responses.
- Not verifying if a finding is a real vulnerability.
- Skipping documentation and follow-up.

18 Incident Response in Your Home Lab

VERIQTA

1. Understanding the Incident Response Lifecycle

- Incident response is a structured process for handling security incidents.
- It helps you detect, contain, and recover from security events.
- Following a consistent process reduces damage and downtime.
- You can practice the incident response lifecycle in a home lab using realistic scenarios.



2. Detecting, Investigating, Containing, and Recovering

1. **Detect**
   Identify unusual activity using logs, alerts or monitoring tools.
2. **Investigate**
   Analyze the evidence to understand what happened, how it happened and the impact.
3. **Contain**
   Take action to stop the incident and prevent further damage (e.g. block IP, disable account).
4. **Recover**
   Restore normal operations and verify that the environment is secure.

3. Collecting Logs and Preserving Evidence

- Collect relevant logs from system, application and security tools.
- Preserve evidence before making changes.
- Use read-only copies where possible.
- Note the date, time and source of each log.
- Store evidence in a safe location for analysis.



Log Source	Location / Command	What It Shows
Authentication logs	/var/log/auth.log	User logins and failures
System logs	/var/log/syslog	System events
Application logs	/var/log/<app>/	Application activity
Journal logs	sudo journalctl	Service and system events
Web server logs	/var/log/apache2/ or /var/log/nginx/	Web requests and errors

4. Building an Incident Timeline

- Arrange events in chronological order.
- Include date, time, source, and a short description.
- A timeline helps you understand what happened and communicate it clearly.

Time (UTC)	Event	Source	Details
10:02:15	Failed login attempt	/var/log/auth.log	Invalid user from 192.168.56.10
10:05:30	Successful login	/var/log/auth.log	User admin from 192.168.56.10
10:07:12	Suspicious command	/var/log/syslog	User ran privilege escalation tool
10:10:45	Multiple file changes	/var/log/syslog	Modified system files in /etc/
10:15:20	Containment applied	Firewall logs	Blocked IP 192.168.56.10

5. Writing a Beginner Incident Report

- Clearly describe what happened.
- Include the timeline, impact and actions taken.
- Keep it simple, factual and concise.
- Provide recommendations to prevent it in the future.

Sample Report Structure

1. Incident Summary – short description of the incident.
2. Timeline – key events in chronological order.
3. Impact – what was affected.
4. Actions Taken – how you contained and resolved it.
5. Root Cause – what caused the incident.
6. Lessons Learned – what you will do differently.
7. Recommendations – how to prevent it.



6. Verifying That Normal Operation is Restored

- Check that affected services are working correctly.
- Confirm that no unauthorized access remains.
- Monitor logs for continued suspicious activity.
- Test key functionality (e.g. web application, SSH, database).
- Document the final status and close the incident.



Verification Checklist

- ☑ All critical services are running.
- ☑ No active suspicious activity in logs.
- ☑ User accounts are secure (passwords reset if needed).
- ☑ Firewall rules and security controls are in place.
- ☑ Applications are accessible and functioning normally.
- ☑ Final review of logs shows no further issues.

7. Best Practices

- Practice incident response in a safe lab environment.
- Keep detailed and accurate logs.
- Use a consistent process for every incident.
- Communicate findings clearly.
- Review and improve your process after each incident.

8. Common Mistakes to Avoid

- Not collecting logs before making changes.
- Ignoring small suspicious activity.
- Deleting evidence by mistake.
- Not documenting the incident.
- Assuming the system is clean without verification.
- Skipping the lessons learned step.

19 Automating Basic Security Checks with Bash

1. Understanding Security Automation

- Security automation uses scripts to regularly check your system for common security issues.
- It helps you find problems early and keeps your environment more secure.
- Automation saves time and ensures checks are consistent.
- In this lab, you will write a simple Bash script to perform read-only security checks.



2. Writing a Bash Script to Check System State

```
#!/bin/bash
# Basic security check script
TIMESTAMP=$(date +"%Y-%m-%d_%H-%M-%S")
OUTPUT_DIR="$HOME/security-checks"
mkdir -p "$OUTPUT_DIR"
OUTPUT_FILE="$OUTPUT_DIR/check_$TIMESTAMP.txt"

echo "=== System Security Check ===" | tee -a "$OUTPUT_FILE"
echo "Date: $(date)" | tee -a "$OUTPUT_FILE"
echo "Hostname: $(hostname)" | tee -a "$OUTPUT_FILE"
echo "User: $(whoami)" | tee -a "$OUTPUT_FILE"
```

3. Checking Running Services and Open Ports

Check Running Services

List all running services:

```
systemctl list-units
--type=service --state=running
```

Save to file:

```
systemctl list-units --type=service
--state=running >> "$OUTPUT_FILE"
```

Check Open Ports

List open network ports:

```
ss -tulnp
```

Or using netstat:

```
sudo netstat -tulnp
```

Save to file:

```
ss -tulnp >> "$OUTPUT_FILE"
```

4. Recording Results in Timestamped Files

- Create a dedicated directory to store your results.
- Use timestamps in filenames to keep a history.
- Append all check results to the same file.
- This helps you compare results over time.



```
# Create directory and Timestamped file
OUTPUT_DIR="$HOME/security-checks"
mkdir -p "$OUTPUT_DIR"
OUTPUT_FILE="$OUTPUT_DIR/check_$(date +"%Y-%m-%d_%H-%M-%S").txt"
echo "Results saved to: $OUTPUT_FILE"
```

5. Handling Command Failures

- Use error handling to make your script more reliable.
- Check if commands succeed or fail.
- Continue running other checks even if one command fails.
- Log error messages to the output file.



```
echo "Checking firewall status..." | tee -a "$OUTPUT_FILE"
if command -v ufw >/dev/null 2>&1; then
  sudo ufw status | tee -a "$OUTPUT_FILE"
else
  echo "UFW not installed" | tee -a "$OUTPUT_FILE"
fi
```

6. Scheduling Safe, Read-Only Checks

- Use cron to run the script automatically.
- Run only read-only checks (no changes to the system).
- Choose a safe schedule (e.g. once per day).
- Verify that the script runs and results are saved.



```
# Make the script executable
chmod +x ~/security-checks/security-check.sh
# Add to cron (run daily at 9:00 AM)
crontab -e
# Add this line to run the script daily
0 9 * * * /home/youruser/security-checks/security-check.sh
```

7. Best Practices

- Use read-only commands.
- Keep scripts simple and well documented.
- Store results with timestamps.
- Handle errors gracefully.
- Review results regularly.
- Update your script as you learn more checks.
- Keep the script secure (correct permissions).

8. Common Mistakes to Avoid

- Running commands that make system changes.
- Not handling command failures.
- Overwriting previous results.
- Using scripts without testing.
- Scheduling checks too frequently.
- Ignoring error messages.
- Storing the script in an insecure location.
- Not reviewing the results.

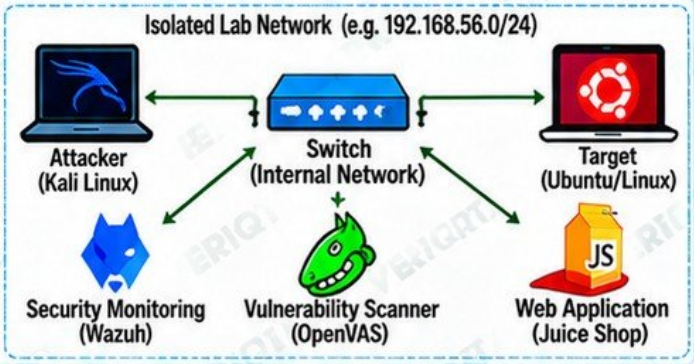
20 Your Complete Cybersecurity Home Lab Project

VERIQTA

1. Lab Overview and Goals

- Build a complete cybersecurity home lab with isolated network, Linux machines and security tools.
- Run an authorized network assessment.
- Capture and investigate security events.
- Review firewall rules and system hardening.
- Document findings, remediation and verification.
- Prepare a professional portfolio project for GitHub.

2. Lab Architecture Diagram



3. Setting Up the Lab Environment

1. Create an isolated network (e.g. using VirtualBox/VMware).
2. Install Linux machines (Kali and Ubuntu).
3. Install security tools (Wazuh, OpenVAS, required packages).
4. Configure network settings and verify connectivity.
5. Keep the lab isolated from your main network and the internet.

```
# Verify network connectivity
ip a
ping 192.168.56.10
# Check isolation (should NOT reach internet)
ping 8.8.8.8  # This should fail in isolated network
```

4. Running an Authorized Network Assessment

1. Use Nmap to discover live hosts.
2. Scan for open ports and services.
3. Use a vulnerability scanner (OpenVAS).
4. Review and understand the findings.

```
# Nmap scan
nmap -sS -sV -O 192.168.56.0/24
# OpenVAS scan (example using gvm-cli)
gvm-cli socket --gmp "create target name=Lab-Target
hosts=192.168.56.10"
gvm-cli socket --gmp "create task target=Lab-Target"
gvm-cli socket --gmp "start task <task-id>"
```

5. Capturing and Investigating Security Events

1. Generate test events (e.g. failed login attempts, port scans, suspicious activity).
2. Collect and analyze logs (auth logs, system logs, Wazuh alerts).
3. Investigate events using timestamps and evidence.
4. Build a timeline of what happened.

```
# View authentication logs
sudo journalctl -u ssh --since "2026-10-14"
# Check Wazuh alerts
sudo tail -f /var/ossec/logs/alerts/alerts.json
```

6. Reviewing Firewall Rules and System Hardening

1. Check current firewall rules (UFW).
2. Verify open and closed ports.
3. Review system hardening settings.
4. Fix any issues found in the assessment.

```
# Check UFW rules
sudo ufw status numbered
# Check open ports
ss -tulnp
# Check running services
systemctl list-units --type=service --state=running
```

7. Documenting Findings and Remediation

1. Summarize the vulnerabilities and security issues found.
2. Document the steps you took to fix them.
3. Verify that the issues are resolved.
4. Include screenshots, commands and results.

8. Preparing a Portfolio Project for GitHub

1. Create a clear project structure and README.
2. Include your lab architecture diagram.
3. Document tools used, key findings and remediation.
4. Explain what you learned and next steps.

9. Final Verification Checklist

- All machines are running and connected.
- Network assessment completed.
- Security events captured and investigated.
- Firewall rules and hardening verified.
- All findings documented with evidence.
- Project is ready for GitHub.

10. Common Mistakes to Avoid

- Connecting the lab to your main network or the internet.
- Running scans without authorization (only scan your own lab).
- Not properly documenting your steps and findings.
- Ignoring security alerts.
- Skipping verification after remediation.
- Having an unclear or incomplete GitHub project.